

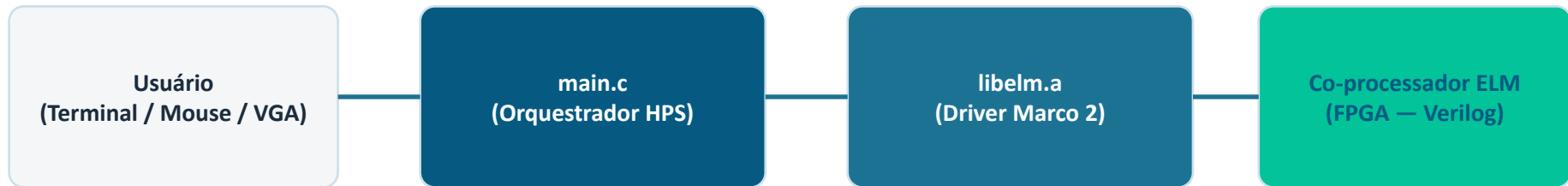
Marco 3

API C para o Classificador de imagem via IP-Core VGA

Rogério Cerqueira
Jones Bacelar

Comunicação do Sistema

HPS (ARM Cortex-A9) ↔ FPGA via PIOs na Lightweight Bridge (MMIO)



PIOs criados no Platform Designer (Qsys) — Lightweight HPS-to-FPGA Bridge

data_in	0xFF200000	32 bits	HPS escreve instrução codificada → co-processador lê
data_out	0xFF200010	32 bits	Co-processador escreve resultado + flags (DONE/BUSY/ERROR/pred)
ctrl	0xFF200020	3 bits	HPS escreve: bit0=ENABLE, bit1=CLR_OP, bit2=RESET
vga_addr	0xFF200070	10 bits	HPS escreve endereço do pixel no framebuffer (0..783)
vga_color	0xFF200080	9 bits	HPS escreve cor RRRGGGBB do pixel
vga_ctrl	0xFF200090	3 bits	HPS escreve: bit0=ENABLE, bit1=WRITE_EN, bit2=RST
vga_status	0xFF2000A0	1 bit	FPGA responde DONE quando framebuffer concluiu a escrita

Modo 1 — Inferência por Arquivo (.mif)

main.c chama `modo_arquivo(elm, vga, cfg)` após inicializar ELM e VGA

1

`elm_mif_load_image(path, &img, &sz)`

Lê arquivo .mif, valida dimensões (WIDTH=8, DEPTH=784) e retorna array `uint8_t` com escala de cinza 0–255.

2

`vga_draw_image(vga, img)`

Itera sobre 784 pixels: converte para 9 bits (RRRGGBB), envia via PIOs (`vga_addr/color`) e aguarda sinal DONE.

3

`elm_reset(elm)`

Reseta a FSM do hardware via PIO ctrl (RESET/CLR_OP). Limpa acumuladores sem apagar pesos nas BRAMs.

4

`elm_classify(elm, img, &pred)`

Envia imagem (STORE_IMG) e inicia processamento (START). O IP-Core executa a rede neural completa na FPGA.

5

`data_out[3:0] → pred`

HPS lê registrador de saída: extrai dígito predito (bits 0-3) e verifica flags de status (DONE/BUSY/ERROR).

Modo 2 — Desenho com Mouse

main.c chama modo_desenho(elm, vga, cfg). Leitura via /dev/input/event0 (Linux Input API)

Leitura de Eventos do Kernel

Cursor em Cruz + Inferência

EV_REL / REL_X,Y

Movimento relativo → acumula posição VGA (0–639, 0–479) → converte para canvas 28×28 dividindo pela escala 10×

BTN_LEFT (press)

Pinta canvas[idx]=255 + 4 vizinhos=255. Chama vga_set_pixel() para cada pixel modificado

BTN_MIDDLE(press)

Apaga canvas[idx]=0 + 4 vizinhos=0. Restaura pixels para VGA_BLACK

BTN_RIGHT (press)

Confirma desenho. Sai do loop de captura e envia canvas ao co-processador

cursor_draw()

Pinta 5 pixels (centro + 4 vizinhos) em VGA_RED. Só sobrepõe pixels pretos (canvas=0)

cursor_erase()

Restaura cor original: canvas≠0 → VGA_WHITE, canvas=0 → VGA_BLACK

Escala 10×

Canvas 28×28 → tela 280×280 centralizada em 640×480 (offset X=180, Y=100)

Após BTN_RIGHT

elm_reset(elm) → elm_classify(elm, canvas, &pred) → exibe predição

Modo 3 — Benchmark com PNGs do MNIST

Avalia o classificador em imagens reais. Lê PNGs diretamente via `stb_image` (sem pré-conversão)

- 1 **listar_pngs():** lê pasta `test/<digito>/`, ordena alfabeticamente → determinístico
- 2 **carregar_png():** `stb_image` lê PNG → `uint8_t[784]` grayscale, redimensiona se $\neq 28 \times 28$
- 3 **vga_draw_image():** exibe a imagem no monitor durante a inferência
- 4 **elm_reset():** zera FSM do co-processador
- 5 **clock_gettime() → elm_classify():** mede latência e executa inferência
 $\text{lat} = (\text{t1.tv_sec} - \text{t0.tv_sec}) \times 1000 + (\text{t1.tv_nsec} - \text{t0.tv_nsec}) / 1e6 \text{ [ms]}$
- 6 **Gravação de Resultados:** indice, arquivo, esperado, predito, correto, `latencia_ms`

Métricas do Benchmark — Como são Calculadas

Calculadas após processar todas as N imagens. Salvas no arquivo .log junto ao CSV.

Acurácia

$$\text{acurácia} = (n_{\text{acertos}} / n_{\text{válidos}}) \times 100\%$$

n_{acertos} : imagens onde $\text{pred} == \text{digito esperado}$. $n_{\text{válidos}}$: imagens processadas com sucesso (exclui falhas de leitura).

Latência Média

$$\text{média} = \sum \text{lat}_i / n$$

Soma de todas as latências individuais dividida pelo número de inferências. Cada latência mede apenas `elm_classify()`, excluindo VGA e I/O.

Desvio Padrão

$$\sigma = \sqrt{(\sum (\text{lat}_i - \text{média})^2 / n)}$$

Mede a dispersão das latências. σ pequeno $\rightarrow$ inferências consistentes. σ grande $\rightarrow$ variações de timing (cache miss, IRQ, etc.).

Mínima / Máxima

$$vmin = \min(\text{lat}_i) \quad vmax = \max(\text{lat}_i)$$

Melhor e pior caso individual. Útil para detectar outliers — picos podem indicar interferência do SO ou miss de cache da BRAM.

Throughput

$$\text{throughput} = 1000 / \text{média} \text{ [imagens/segundo]}$$

Quantas inferências por segundo o co-processador consegue processar em média. Derivado diretamente da latência média em ms.

Correções e Decisões Técnicas

Problemas encontrados durante o desenvolvimento e como foram resolvidos

elm_reset() antes de cada inferência

FSM da FPGA acumulava resíduos → todas as inferências prediziam o mesmo dígito. reset() pulsa ELM_SIG_RESET + CLR_OP no PIO ctrl.

elm_init_weights() — pesos nunca carregados

elm_open() só mapeava registradores. Sem carregar W_in/bias/beta, BRAMs continham zeros → pred constante = 0.

GCC 4.6.3 na placa (-std=gnu99)

GCC antigo não suporta -std=c11 nem __isoc23_strtol (glibc moderna). Makefile do Marco 2 corrigido; libelm.a recompilada nativamente na placa.

Divisão exata por 10 no ghrd_top.v

$cx = (dx \gg 4) + (dx \gg 5) + (dx \gg 8) \approx dx \times 0.097$ (erro acumulado). Corrigido para $cx = dx/10$. Garante mapeamento correto pixel VGA → canvas 28×28.

stb_image no benchmark

Eliminou pré-conversão de 1000+ PNGs para .mif. Leitura + conversão grayscale 28×28 feita em tempo real no ARM, sem dependência externa.

Permissão de execução do binário

cp -r da pasta de backup não preservou o bit +x. Binário existia mas sudo ./app retornava 'command not found'. Corrigido com chmod +x app.

Conclusão



Integração C & Hardware

API C completa: main.c orquestra ELM (Marco 2), VGA e mouse via MMIO.



Comunicação HPS-FPGA

7 PIOs criados no Platform Designer via Lightweight Bridge.



Modos de Operação

Suporte a Arquivo (.mif), Desenho livre e Benchmark automatizado.



Análise de Performance

Métricas precisas de acurácia, latência e throughput via clock_gettime.